Teste de Desenvolvedor Java Pleno/Sênior - Sistema de Pedidos Desktop Assíncrono

Stacks Envolvidas:

- **Linguagem:** Java
- **Backend/Serviço Principal:** Spring Boot
- **GUI:** Java Swing
- **Mensageria:** RabbitMQ

Tempo Máximo: 3 horas

Objetivo: Desenvolver uma aplicação desktop Java Swing que envia "pedidos" para um serviço Spring Boot. O serviço, por sua vez, processa esses pedidos de forma assíncrona via RabbitMQ e envia atualizações de status de volta, que a aplicação Swing deve exibir.

Requisitos e Tarefas:

1. Serviço Backend (Spring Boot) - (~1h30min)

Este será o componente responsável por receber os pedidos, orquestrar a mensageria e simular o processamento.

- **Configuração Inicial:**
  - Crie um projeto **Spring Boot**.
  - Adicione as dependências necessárias para **Spring Web**, **Spring AMQP** (para RabbitMQ) e **Jackson** (para serialização/desserialização JSON).
  - Configure a conexão com um servidor RabbitMQ (utilizar o servidor disponibilizado no final desse arquivo, ou “dockerizado”).
- **Endpoint HTTP:**
  - Crie um endpoint REST (`POST /api/pedidos`) que aceite um payload JSON representando um `Pedido` (com `id` (UUID), `produto` (String), `quantidade` (int) e `dataCriacao` (LocalDateTime)).
  - **Validação Simples:** Adicione uma validação básica (ex: `quantidade` maior que zero, `produto` não vazio). Em caso de falha, retorne HTTP 400.
- **Publicação no RabbitMQ:**
  - Se o pedido for válido, publique o objeto `Pedido` (serializado para JSON) em uma fila do RabbitMQ chamada `pedidos.entrada.seu-nome`.

    - Após a publicação bem-sucedida, retorne um status HTTP 202 Accepted, incluindo o `id` do pedido, indicando que a requisição foi recebida e será processada assincronamente.
- **Consumo e Processamento Assíncrono:**
    - Crie um **consumidor** para a fila `pedidos.entrada.seu-nome`.
    - Ao receber um `Pedido`:
        - Simule um tempo de processamento de 1-3 segundos (`Thread.sleep()`).
        - Gere um número aleatório. Se o número for menor que 0.2 (20% de chance), simule uma `ExcecaoDeProcessamento`.
        - Logue o início e o fim do processamento do pedido (sucesso ou falha).
- **Publicação de Status de Retorno:**
    - **Sucesso:** Se o processamento for bem-sucedido, publique uma mensagem de `StatusPedido` (com `idPedido`, `status: SUCESSO`, e `dataProcessamento`) em uma fila do RabbitMQ chamada `pedidos.status.sucesso.seu-nome`.
    - **Falha (DLQ/Rejeição):** Se ocorrer a `ExcecaoDeProcessamento`:
        - **Rejeite a mensagem original** da fila `pedidos.entrada.seu-nome` de forma que ela seja direcionada para uma **Dead Letter Queue (DLQ)**. O candidato deve demonstrar como configurar a fila `pedidos.entrada.seu-nome` para ter uma DLQ associada (ex: `pedidos.entrada.seu-nome.dlq`).
        - Publique também uma mensagem de `StatusPedido` (com `idPedido`, `status: FALHA`, e `mensagemErro`) em uma fila chamada `pedidos.status.falha.seu-nome`.
- **Manutenção de Status em Memória (Opcional, mas útil):**
    - Para o teste de 3h, o backend pode manter um `Map<UUID, String>` em memória para armazenar o status atual de cada pedido (recebido, processando, sucesso, falha). Isso será útil para o polling do Swing.
    - Adicione um endpoint GET simples (`/api/pedidos/status/{id}`) para que o Swing possa consultar o status de um pedido específico.

2. Aplicação Desktop (Java Swing) - (~1h30min)

Esta será a interface gráfica que interage com o backend.

- **Configuração Inicial:**
    - Crie um projeto Java simples (pode ser Maven ou Gradle).
    - Adicione as dependências para `HttpClient` (ex: OkHttp ou Spring `RestTemplate` se o candidato preferir, ou mesmo `java.net.HttpURLConnection` para o básico).

    - Adicione as dependências para **Jackson** (para serialização/desserialização JSON).
- **Interface do Usuário (GUI):**
    - Uma janela Swing (`JFrame`) com:
        - Campos de texto (`JTextField`) para `produto` e `quantidade`.
        - Um botão "Enviar Pedido".
        - Uma área de texto não editável (`JTextArea` ou `JTable`) para exibir o `id` e o status de cada pedido enviado. Cada nova linha deve conter o `id` e o status atual.
- **Envio de Pedidos:**
    - Ao clicar em "Enviar Pedido", a aplicação deve:
        - Criar um objeto `Pedido` (gerando um `UUID` para o `id`).
        - Enviar uma requisição POST para o endpoint do backend (`/api/pedidos`) com o `Pedido` serializado em JSON.
        - Imediatamente após o envio, adicione uma entrada na área de exibição, mostrando o `id` do pedido e um status inicial "ENVIADO, AGUARDANDO PROCESSO".
- **Atualização de Status (Polling Assíncrono):**
    - **Crucial para o Swing:** Implemente um mecanismo de **polling assíncrono** para consultar o status dos pedidos.
    - Crie um `SwingWorker` ou `Timer` que, a cada poucos segundos (ex: 3-5 segundos), itere sobre os pedidos que estão "AGUARDANDO PROCESSO" na sua lista interna.
    - Para cada pedido, faça uma requisição GET para o endpoint de status do backend (`/api/pedidos/status/{id}`).
    - Se o status retornado for "SUCESSO" ou "FALHA", atualize a linha correspondente na área de exibição do Swing. Certifique-se de que as atualizações da GUI ocorram na Event Dispatch Thread (EDT) usando `SwingUtilities.invokeLater()`.
- **Tratamento de Erros no Cliente:**
    - Exiba mensagens de erro amigáveis ao usuário caso a requisição para o backend falhe (ex: servidor offline, validação falhou).

O que vamos avaliar:

1. **Proficiência em Java (Senior):**
    - Uso de POJOs, princípios de orientação a objetos, tratamento de exceções.
    - Conhecimento de estruturas de dados básicas.
2. **Spring Boot:**

   - Criação de APIs REST, uso de `@RestController`, `@PostMapping`, `@GetMapping`.
   - Injeção de dependência e componentes Spring.
   - Configuração e uso do Spring AMQP (`@RabbitListener`, `RabbitTemplate`).
3. **RabbitMQ:**
   - Correção na publicação e consumo de mensagens.
   - Entendimento e configuração de **Dead Letter Queues (DLQ)** para lidar com falhas de processamento.
   - Serialização/desserialização de mensagens.
4. **Java Swing:**
   - Criação de interfaces gráficas básicas (`JFrame`, `JButton`, `JTextField`, `JTextArea`).
   - **Programação Assíncrona com Swing:** Uso correto de `SwingWorker` ou `Timer` para não bloquear a GUI e `SwingUtilities.invokeLater()` para atualizações da interface.
   - Manuseio de eventos (listeners de botão).
5. **Comunicação Inter-processos:**
   - Integração eficaz entre o cliente Swing (via HTTP) e o backend (via HTTP e RabbitMQ).
6. **Qualidade do Código:**
   - Organização do projeto, legibilidade, boas práticas de codificação.
   - Tratamento de erros robusto e logging adequado em ambos os componentes.
7. **Testes Unitários:** Escrever o teste unitário da função que publicará a mensagem na fila**.**

Observações para o Candidato:

- Não é necessário configurar um banco de dados. Os status e dados dos pedidos podem ser mantidos em memória no backend.
- Assuma que um servidor RabbitMQ já está disponível e acessível no endereço: amqp://jaragua-01.lmq.cloudamqp.com (usuário: bjnuffmq, senha: gj-YQIiEXyfxQxjsZtiYDKeXIT8ppUq7)
- https://jaragua-01.lmq.cloudamqp.com (Gerenciamento, mesmas credenciais)
- Foco na funcionalidade, integração e nas boas práticas das tecnologias envolvidas. A beleza da GUI não é o principal foco, mas a usabilidade básica sim.
